HealthMate
Viva from Zero

A beginner's learning and rehearsal guide

Kotlin + Jetpack Compose + Room
Prepared for the Navod718/healthmate project
10 November 2026

Start here
No previous mobile-development knowledge is assumed. Learn the vocabulary, run each feature, trace its code, then answer aloud in your own words.

Inside: Android basics, Kotlin essentials, your architecture and database, source walkthroughs, real testing evidence, 50 viva questions, coding exercises and a guided demonstration.

This is a study guide. It is not your final coursework report and does not prove individual authorship. Explain the assistance you actually used and demonstrate your own understanding.

Repository: github.com/navod718/healthmate

Version note: the merged core app and the optional PR #3 additions are distinguished throughout. Repository status was checked when this guide was prepared.

1. How to use this guide

A viva is a conversation in which the lecturer checks whether you understand the application, can explain decisions and can connect visible behavior to source code. A memorised definition is much stronger when you can demonstrate it.

Pass	What to do	Your checkpoint
1: Vocabulary	Read chapters 3-7. Say each term without looking.	Explain Kotlin, Compose, Room and an APK.
2: Follow the code	Read chapters 8-20 with Android Studio open.	Trace one Save action through all layers.
3: Explain behavior	Read chapters 21-26 and perform the demo.	Show persistence, filtering and validation.
4: Rehearse	Use the questions, coding exercises and mock viva.	Answer, show the file, explain one edge case.

A simple answer pattern

What it is -> where it appears -> why it is used -> one limitation or example. For Room: it is an Android library over SQLite; our entity/DAO/database use it; it reduces manual database code and checks supported queries during compilation; migrations still need deliberate design.

If you only have one evening

Prioritise the stack, architecture, Add/Edit/Delete flow, validation, state restoration, database migration, tests and the demo. Study optional trends only if the version you will demonstrate contains them.

Try it: Create a notebook with three columns: term, explanation in your own words, and the HealthMate file that demonstrates it. Tick a topic only after you can show it.

2. Contents

Use these links or the PDF bookmarks to jump to a topic. Read the early chapters in order the first time.

1. How to use this guide 2
2. Contents 3
3. What your mobile app actually does 5
4. Your technology stack and project versions 6
5. Kotlin basics: values and types 7
6. Kotlin basics: functions and data classes 8
7. Kotlin reading skills: nulls, lists and lambdas 9
8. The Android Activity lifecycle 10
9. How the app starts 11
10. Jetpack Compose and recomposition 12
11. Layout, navigation and keyboard behavior 13
12. MVVM: the architecture you must explain 14
13. Your source-code map 15
14. StateFlow, events and reactive updates 16
15. Coroutines, suspend and the main thread 17
16. Database fundamentals and your two tables 18
17. DAO and SQL: read the important queries 19
18. Walkthrough: one Add Record action 20
19. Validation: rules and reasoning 21
20. History, date filtering and daily totals 22
21. Edit and Delete without losing identity 23
22. Goals, progress and database migration 24
23. Drafts: rotation, process death and limits 25
24. Optional PR #3: trends and quick water 26
25. Testing: what the evidence actually says 27
26. Git, GitHub and continuous integration 28
27. Build settings, APKs and release signing 29
28. Errors, privacy and honest limitations 30
29. Your guided demonstration 31
30. Small live-coding exercises 32
31. Viva questions 1-10: foundations 33

32. Viva questions 11-20: architecture and state 34
33. Viva questions 21-30: data and CRUD 35
34. Viva questions 31-40: behavior and testing 36
35. Viva questions 41-50: extras and judgement 37
36. Mock viva: rehearse a real conversation 38
37. Five-day revision and readiness checklist 39
38. Glossary: essential terms A-M 40
39. Glossary: essential terms N-Z 41
40. Official references and final reminders 42

3. What your mobile app actually does

A mobile app is software running on a phone or tablet operating system. HealthMate is a **native Android app**: it uses Android libraries and runs on Android. Kotlin alone does not make this project an iPhone app.

HealthMate lets a user manually record steps walked, calories burned and water intake. It stores dated records locally, displays today's totals, supports search by date, and allows changes or deletion. Daily goals add personal progress targets.

Important distinction
Steps and calories are manually entered. There is no pedometer, sensor integration, smartwatch link or medical calculation. Water quick-entry, when present, is also a manual logging action.

Term	Plain meaning
Android	The operating system on the target device.
Emulator	A virtual Android device running on a computer.
APK	An installable Android application package.
Offline	The implemented features do not require a network service.
Persistence	Saved data survives closing and reopening the app.

Say it aloud
Describe HealthMate in 20 seconds.
HealthMate is an offline Android wellness tracker built with Kotlin, Jetpack Compose and Room. Users manually log dated activity, review daily totals and goals, filter their history, and edit or delete records.

Try it: Open the app, locate Dashboard and Records, and explain which values come from user input.

4. Your technology stack and project versions

Technology	Its job	Do not confuse it with
Kotlin	Programming language for logic and UI definitions.	Android Studio, which is an editor/IDE.
Jetpack Compose	Android toolkit for describing UI in Kotlin.	Room, which manages persistence.
Room	Android persistence library over SQLite.	A remote server or a cloud database.
SQLite	Embedded relational database engine.	A network API.
Gradle / KSP	Build/dependency tasks / code processing for Room.	Application features the user taps.

Verified main: commit 0b3377d contains the merged core app, goals, drafts and records management. Its feature snapshot had 15 JVM + 15 emulator tests passing.

Optional additions: PR #3, commit fab2f19 on feature/trends-quick-water, adds weekly charts and quick water. It had 20 JVM + 19 emulator tests passing. PR #3 was still open and unmerged on 10 November 2026. Check the branch before claiming those extras are in your demo.

The original draft brief names sqflite; the project follows your reported lecturer instruction to use Kotlin, Compose and Room. Describe that clarification accurately and keep the lecturer's evidence.

Say it aloud
Why this stack?
The updated requirements specified Kotlin, Compose and Room. They provide native Android development, declarative UI and local SQLite persistence with coroutine and Flow support.

5. Kotlin basics: values and types

A variable is a named value. A type describes what kind of value it can contain. Kotlin often infers the type from the assigned value.

```
val water: Int = 500
var selectedDate: String = "2026-11-06"
val totalWater: Long = 2500L
val saving: Boolean = false
val progress: Float = 0.5f
val label = "$water ml"
```

• **val:** the reference/value cannot be reassigned. A val referring to a mutable object does not make that object deeply immutable.

• **var:** can be reassigned. UI state still needs an observable state holder for Compose to notice changes.

• **Int:** a 32-bit whole number. Our individual health inputs fit 0 to 2,147,483,647.

• **Long:** a 64-bit whole number. Record ids and summed totals use it.

• **String, Boolean, Float:** text, true/false and decimal progress respectively.

The form initially holds text, even for numbers. A user may have typed nothing or an invalid number. Validation checks the text before conversion to Int.

Say it aloud
Why use Long for totals?
Each entry is an Int, but adding valid entries can exceed the Int limit. Room maps the aggregate to Long so ordinary multi-entry sums do not overflow a 32-bit total.

Try it: Predict the values after: val a = 250; val b = 500; val total = a + b. Answer: 750. Then explain why an empty String is not automatically the number zero.

6. Kotlin basics: functions and data classes

A function groups an action or calculation. A class defines a kind of object. A data class is convenient for values because Kotlin generates equality, copy and readable string behavior.

```
// Teaching examples
fun addWaterAmounts(first: Int, second: Int): Int {
    return first + second
}

data class ExampleRecord(val steps: Int, val water: Int)
val original = ExampleRecord(1200, 500)
val edited = original.copy(water = 750)
```

original still contains 500 ml. edited contains 750 ml. This copy pattern is important: the ViewModel publishes a new state value instead of quietly changing a field in the old state object.

An **interface** specifies operations another implementation must provide. HealthRecordDao is an interface; Room generates its database implementation. Fake DAOs can implement the same contract in unit tests.

An **annotation** begins with @ and adds information for tooling/frameworks: @Composable marks a UI function, @Entity describes a Room table and @Query associates SQL with a DAO method.

Open in Android Studio: app/src/main/java/com/navod718/healthmate/feature/health/data/HealthRecord.kt
Open verified source snapshot on GitHub

Say it aloud
What is HealthRecord?
It is a Kotlin data class annotated as a Room entity. One object represents one dated activity entry, and Room maps its properties to columns in health_records.

7. Kotlin reading skills: nulls, lists and lambdas

```
val recordId: Long? = null       // nullable value
val id = recordId ?: 0L          // fallback if null
val text: String? = null
val length = text?.length       // safe access; result is null

val values = listOf(250, 500)
val total = values.sum()         // 750
val labels = values.map { "$it ml" }
```

? permits null. **?.** accesses a member only when the receiver is non-null. **?:** is the Elvis operator, supplying a fallback. **!!** asserts non-null and can crash if wrong; do not suggest it as a general fix.

A **lambda** is a small function written in braces. In map, it is the current element. UI callbacks such as onClick = { model.save() } tell the interface what to do after a user action.

Lists contain ordered values; maps store key/value pairs. Validation returns a Map<String, String>: for example, steps maps to an error message. A null date filter means show all dates.

== checks equality; != means not equal; && means both conditions; || means either; ! reverses a Boolean. A suspend function may pause without blocking its thread. It is not automatically a background-thread function.

Say it aloud
How is Add distinguished from Edit?
EditorState.id is nullable. Null means create a new record; an existing id means update that specific record.

8. The Android Activity lifecycle

The lifecycle is the sequence of callbacks Android uses as an Activity is created, becomes visible, receives interaction and leaves the screen. You do not control when the operating system may reclaim the app process.

Callback	Beginner explanation
onCreate	Initial setup. MainActivity establishes Compose content here.
onStart	The Activity becomes visible.
onResume	The Activity is ready for foreground interaction. HealthMate refreshes today on this event.
onPause	The Activity is losing foreground focus; keep this callback lightweight.
onStop	The Activity is no longer visible.
onDestroy	The Activity is being destroyed, but this callback is not guaranteed before process death.

```
Typical opening: onCreate -> onStart -> onResume
Leaving the screen: onPause -> onStop
Returning to a stopped Activity: onRestart -> onStart -> onResume
```

Rotation normally recreates the Activity for a configuration change. A ViewModel can be retained through this recreation. A killed process needs new objects later, so saved-state restoration and committed database persistence have different jobs.

Do not wait until onDestroy to save important health records. HealthMate writes a record when the user successfully saves it. Composable recomposition is also different from Activity recreation: recomposition can occur many times while one Activity remains running.

Say it aloud
Why collect state with the lifecycle?
It aligns collection with the active Android UI lifecycle. It is different from running a permanent background service, and helps avoid collecting unnecessarily when that lifecycle is inactive.

Try it: Press Home in the emulator, then return to HealthMate. Compare this with rotating the emulator and with uninstalling it. Explain why those three actions affect data/state differently.

9. How the app starts

Android starts the application process when needed. The manifest identifies HealthMateApplication and the launcher activity. The app uses one Activity containing multiple Compose screens.

• **AndroidManifest.xml:** declares the application, MainActivity and MAIN/LAUNCHER intent filter. Exported true allows the launcher activity to be started externally.

• **HealthMateApplication:** lazily creates the Room database and shared repository. by lazy delays construction until first access; this is not a permanent background service.

• **MainActivity.onCreate:** establishes the Compose content and theme.

• **ViewModel factory:** supplies the repository and a SavedStateHandle using creation extras.

• **HealthMateApp:** chooses the screen, observes state, shows navigation and dialogs.

```
// Core startup shape, abridged from MainActivity
setContent {
    HealthMateTheme {
        // The actual source obtains the ViewModel using a factory.
        HealthMateApp(model)
    }
}
```

Open in Android Studio: app/src/main/java/com/navod718/healthmate/MainActivity.kt
Open verified source snapshot on GitHub

Say it aloud
Is each screen an Activity?
No. This project uses one MainActivity. Dashboard, Records and the shared Add/Edit form are composable functions selected by application state.

10. Jetpack Compose and recomposition

Compose is **declarative**: describe the UI for the current state. When observed state changes, Compose can rerun the relevant UI functions. That update is called recomposition.

```
// Teaching example
@Composable
fun WaterSummary(water: Long) {
    Text("$water ml")
}
```

A composable can receive data and event callbacks. The real RecordForm receives EditorState, a busy flag, onChange and onSave. It renders fields and sends actions upward; it does not execute database SQL itself.

State tools you should distinguish

Tool	Purpose
remember	Keeps a value across recompositions while its composition remains.
rememberSaveable	Can restore supported small UI values through saved-instance state.
ViewModel	Holds screen state and actions outside composables; survives normal configuration recreation.
Room	Persists actual records/goals in a database.

Do not put insert calls directly in a composable body: recomposition could repeat them. Use event callbacks and ViewModel actions. LaunchedEffect is used for controlled coroutine side effects, such as displaying a snackbar.

Say it aloud
Does recomposition restart the app?
No. It updates relevant Compose UI from changed state. It does not mean restarting the process or recreating every screen.

11. Layout, navigation and keyboard behavior

Component	What it does here
Column / Row / FlowRow	Arrange content vertically, horizontally, or with wrapping.
Scaffold	Provides slots for top bar, bottom navigation, content and FAB.
LazyColumn	Composes history items as needed, using stable record-id keys.
OutlinedTextField	Collects text and displays labels, units and errors.
AlertDialog / Snackbar	Confirms choices / gives short feedback.
Modifier	Adds layout, styling, scrolling, semantics and input behavior.

Our navigation is state-based: the tab value chooses Dashboard or Records; editor.open selects the shared form. There is no Navigation Compose NavController in this implementation. Back from Records returns to Dashboard; Back from a changed form asks for confirmation.

KeyboardOptions requests a numeric keyboard, but validation remains necessary because pasted/input-method text may still be invalid. FocusRequester moves Next from steps to calories to water. Done clears focus and hides the keyboard; it does not save automatically.

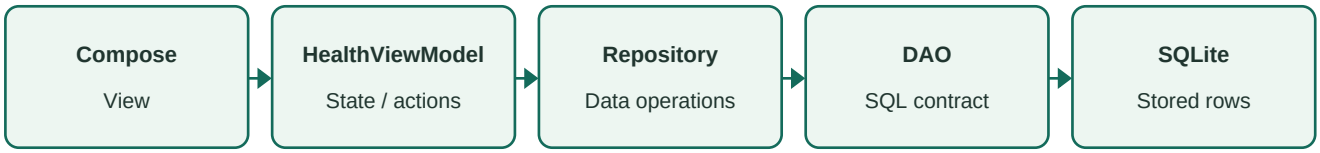
imePadding and vertical scrolling help keep the form usable with the keyboard. Your manual screenshot confirmed numeric input; manual scrolling and automated IME-focus tests provide different kinds of evidence.

Open in Android Studio: app/src/main/java/com/navod718/healthmate/feature/health/presentation/Screens.kt
Open verified source snapshot on GitHub

Try it: Point to one layout modifier, one callback and one composable in RecordForm. Explain their separate jobs.

12. MVVM: the architecture you must explain

MVVM means Model-View-ViewModel. We use a feature-based arrangement with a repository and DAO in the data layer. The separation keeps SQL out of screens and UI widgets out of database code.



Events/writes move right. Observed database results become UI state.

- **View:** composables show state and report clicks/field changes.
- **ViewModel:** validates, coordinates operations and publishes loading/data/error state.
- **Repository:** offers application-facing data methods and validates at the write boundary.
- **DAO:** defines typed database operations; Room generates their implementation.
- **Models/database:** records, goals, query results and the SQLite file.

The repository is constructed and passed into the ViewModel through a factory. This is manual dependency provision; the project does not use Hilt or another dependency-injection framework.

Say it aloud
Why not put everything in MainActivity?
Separating responsibilities makes code easier to understand, reuse and test. MainActivity sets up the app; screens describe UI; the ViewModel handles actions; data classes, repository and DAO handle persistence concerns.

13. Your source-code map

In Android Studio, expand app/src/main/java/com/navod718/healthmate. The root package groups the application code. File names below match the actual project.

File or folder	Your explanation
MainActivity.kt	Android entry point, theme, ViewModel factory.
HealthMateApplication.kt	Lazy Room database and shared repository.
core/database/HealthDatabase.kt	Tables, database version and migration.
core/validation/RecordValidator.kt	Shared text/date validation.
feature/health/data/HealthRecord.kt	Entity and aggregate/result data classes.
feature/health/data/HealthRecordDao.kt	Insert/read/update/delete/filter/totals SQL.
feature/health/data/HealthRepository.kt	Data operations, sample loading, optional weekly filling.
presentation/HealthViewModel.kt	StateFlow, saved draft, actions and errors.
presentation/HealthMateApp.kt	Screen switching, tabs and confirmation dialogs.
presentation/Screens.kt	Dashboard, shared form and date picker.
presentation/RecordsScreen.kt	Lazy history list, filters and row actions.
presentation/GoalsDialog.kt	Goal inputs and save action.
presentation/WeeklyTrends.kt	Optional PR #3 chart; not in verified main.
ui/theme/Theme.kt	Shared Material 3 colours and typography.

app/src/test contains JVM tests. app/src/androidTest contains instrumented Android tests. app/schemas contains exported Room schema JSON. app/build contains generated output and is ignored by Git.

Try it: Find each bold concept from the architecture page in a real file. Do not try to memorise every import.

14. StateFlow, events and reactive updates

A Flow is a stream that can emit values over time. A StateFlow holds a current state value and lets observers receive updates. MutableStateFlow is writable; the ViewModel usually exposes a read-only view with asStateFlow.

```
private val _busy = MutableStateFlow(false)
val busy = _busy.asStateFlow()

// In HealthMateApp:
val editor by model.editor.collectAsStateWithLifecycle()
```

The UI sends an **event**, such as Save. The ViewModel changes **state**, such as busy=true. The UI observes it and displays a saving label or disables controls. Room emits updated query results when relevant tables change; those results become new UI state.

records combines the selected filter with a retry counter. flatMapLatest switches to the newest date query and cancels the previous stream collection. combine joins records with total count; map converts query output into a state object.

stateIn shares an upstream stream as StateFlow. WhileSubscribed(5_000) keeps it running briefly after the last collector disappears; it does not mean poll the database every five seconds. collectAsStateWithLifecycle collects according to the UI lifecycle.

Current HealthMateApp observes these flows at the app-shell level, so a hidden tab's stream may remain collected while that shell is active. Do not claim collection automatically stops for every hidden tab.

Say it aloud
Why do totals refresh after deletion?
The DAO query returns Flow. Room invalidates relevant queries after the table changes, the ViewModel emits new totals, and Compose redraws the values it observes.

15. Coroutines, suspend and the main thread

The main/UI thread processes interaction and drawing. Blocking it with long work makes the app feel frozen. Coroutines provide structured asynchronous work; a suspend function can pause without blocking its current thread.

```
// Shape used by save/delete/goal actions
_busy.value = true
viewModelScope.launch {
    try {
        repository.add(input)
    } catch (e: Exception) {
        if (e is CancellationException) throw e
        // Publish feedback; retain useful user input.
    } finally {
        _busy.value = false
    }
}
```

Room supports suspend database methods and Flow queries, using its asynchronous execution machinery. The word suspend alone does not select Dispatchers.IO for arbitrary work. CPU-heavy custom calculations would need an appropriate dispatcher.

viewModelScope is cancelled when the ViewModel is cleared. Coroutine cancellation should propagate: the code rethrows CancellationException instead of presenting cancellation as an ordinary database failure.

The busy guard blocks another write while one is outstanding. finally resets it on success or failure. A disabled button improves UX; the ViewModel guard also protects the action if called again before the UI redraws.

Open in Android Studio: app/src/main/java/com/navod718/healthmate/feature/health/presentation/HealthViewModel.kt
Open verified source snapshot on GitHub

Say it aloud
What does launch return?
A Job representing the coroutine, not the database result. Inside that coroutine, a suspend call can return its own typed result when complete.

16. Database fundamentals and your two tables

A relational table stores rows with named columns. An entity describes a row type. A primary key uniquely identifies a row. A query requests or changes data using SQL.

health_records column	Kotlin / SQLite	Meaning
id	Long / INTEGER	Auto-generated primary key for one entry.
date	String / TEXT	Canonical date, YYYY-MM-DD.
steps	Int / INTEGER	Non-negative whole steps.
calories	Int / INTEGER	Non-negative whole kcal burned.
water	Int / INTEGER	Non-negative whole ml.

daily_goals column	Meaning
id	Primary key; application convention uses the row id 1.
steps / calories / water	Positive Int targets; defaults are 8000, 400 and 2000.

These are separate tables with no foreign-key relationship. Goals apply to the dashboard generally, rather than being duplicated on each record. Default goals are returned in code if the settings row does not exist; a default row is not automatically inserted.

Multiple records may share a date, so date is not the primary key and is not unique. An index exists on date. SQLite INTEGER storage can represent 64-bit integers; Kotlin Int validation imposes the individual-input limit.

Open in Android Studio: app/src/main/java/com/navod718/healthmate/core/database/HealthDatabase.kt
Open verified source snapshot on GitHub

Say it aloud
Why is id necessary if you already have date?
A date can have several entries. A unique id lets Edit and Delete target one entry without changing every record on that day.

17. DAO and SQL: read the important queries

DAO means Data Access Object. HealthRecordDao describes database operations. @Insert inserts, @Update updates an entity using its primary key, @Query runs SQL and @Transaction groups an operation atomically.

```
-- History query (same SQL, formatted for reading)
SELECT * FROM health_records
WHERE (:date IS NULL OR date = :date)
ORDER BY date DESC, id DESC;

-- Today totals
SELECT COALESCE(SUM(steps), 0) AS steps,
       COALESCE(SUM(calories), 0) AS calories,
       COALESCE(SUM(water), 0) AS water
FROM health_records WHERE date = :date;

-- Delete one row
DELETE FROM health_records WHERE id = :id;
```

- **:date / :id:** bound parameters, not string-concatenated user SQL.
- **DESC:** newest dates first; id descending breaks same-date ties deterministically.
- **SUM:** adds all matching entries. COALESCE converts an empty aggregate result from null to zero.
- **COUNT(*):** counts rows. GROUP BY date, used in optional trends, aggregates per day.

The date index supports date-based access; its exact use depends on the query planner and query shape. ISO dates sort chronologically as text because they are consistently zero-padded and validated.

Open in Android Studio: app/src/main/java/com/navod718/healthmate/feature/health/data/HealthRecordDao.kt
Open verified source snapshot on GitHub

Say it aloud
What does update returning zero mean?
No matching row was updated. The repository/ViewModel treats that as a missing record, rather than inserting a replacement.

18. Walkthrough: one Add Record action

Use this trace whenever the lecturer asks how the layers work together. Example: today, 1200 steps, 60 kcal, 500 ml.

Step	What happens
1. Open	model.add creates EditorState with no id and an initial date. From filtered history, that filter date is the default.
2. Type	RecordForm sends an updated RecordInput through onChange. The ViewModel copies state and stores draft fields in SavedStateHandle.
3. Save	The callback hides the keyboard and calls model.save. Busy/loading guards and validation run first.
4. Write	A coroutine calls repository.add; repository validation checks the boundary and converts Strings to Ints.
5. Persist	dao.insert writes a new health_records row and returns its generated id.
6. Refresh	The form closes after success. Observed queries refresh history, daily totals and optional trends.
7. Failure	A failed write retains the input and shows feedback; finally clears busy.

```
// Actual repository Add logic
require(RecordValidator.validate(input).isEmpty()) {
    "Check the record values."
}
dao.insert(HealthRecord(date = input.date,
    steps = input.steps.toInt(),
    calories = input.calories.toInt(),
    water = input.water.toInt()))
```

Say it aloud
Where is the data saved?
Room writes it into the app's local SQLite file healthmate.db. It is not saved in the composable, GitHub or a cloud account.

19. Validation: rules and reasoning

Validation checks whether input is acceptable before it reaches the database. A numeric keyboard is a convenience, not a validation guarantee.

Input	Actual rule / example
Date	Parse with LocalDate; require canonical YYYY-MM-DD and no future date.
Required fields	Blank steps/calories/water get a field error. Enter 0 for an activity not being logged.
Whole numbers	Only ASCII digits [0-9]+ and a successful toIntOrNull are accepted.
Bounds	Negative, decimal, alphabetic, whitespace-padded and overflowing values are rejected.
Meaningful entry	At least one of steps/calories/water must be greater than zero.

```
// Actual numeric condition, split across lines for reading
if (raw.isBlank()) {
    errors[key] = "This field is required."
} else if (!raw.matches(Regex("[0-9]+")) ||
           raw.toIntOrNull() == null) {
    // Add the whole-number/range error.
}
```

The validator returns error messages keyed by field, with a form-level key for an all-zero entry. Both Add and Edit use it. The repository validates writes again so bypassing the screen does not silently skip the rules.

Open in Android Studio: app/src/main/java/com/navod718/healthmate/core/validation/RecordValidator.kt
Open verified source snapshot on GitHub

Try it: Explain expected results for: empty water, -1 steps, 2.5 kcal, 2147483648 steps, all zero, and water-only 250. The first five fail; water-only 250 passes when the other fields are explicitly 0.

20. History, date filtering and daily totals

RecordsScreen uses LazyColumn and record-id keys to display history. It has distinct loading, database-error, empty-history and no-results-for-this-date states. Those are different situations and need different feedback.

Selecting a date calls filterByDate. The filter is saved and its StateFlow causes flatMapLatest to collect the selected query. Clearing sets null, making the history SQL include every date. This is exact-date filtering, not free-text or date-range search.

Worked example
Two entries on the same day: 1200 steps / 60 kcal / 500 ml and 300 steps / 20 kcal / 250 ml. Dashboard totals are 1500 steps, 80 kcal and 750 ml. History still contains two separate records.

Dashboard uses the device's current local date. refreshToday is called on resume and periodically while the composition is active, so the selected day can change after midnight. This is not a step-counting background service.

When editing moves an entry to another date, today's query stops counting it. A filtered list may also stop showing it; the app tells the user to clear the filter if the saved date is outside it.

Open in Android Studio: app/src/main/java/com/navod718/healthmate/feature/health/presentation/RecordsScreen.kt
Open verified source snapshot on GitHub

Say it aloud
Why did today's totals become zero tomorrow?
The dashboard queries the new local date. Yesterday's data remains stored and is visible in Records; the app does not delete it at midnight.

21. Edit and Delete without losing identity

Edit

The row's Edit action passes its id. The ViewModel shows a loading state, fetches the record, then pre-fills the shared form. EditorState holds both the current input and the original values. An unchanged Edit form can close without a discard prompt.

Save chooses update when id is non-null. The repository constructs a HealthRecord with the same id and calls @Update. It checks the affected-row count. If the row disappeared, the form retains changes and reports that they were not saved; it does not create a duplicate.

Delete

requestDelete selects a target and opens confirmation. Cancel clears the target without writing. Confirm runs delete(id) in a guarded coroutine. A successful delete closes the dialog and observed totals/list update. A database failure leaves a retry/cancel error in the dialog.

```
// Repository behavior
suspend fun delete(id: Long): Boolean = dao.delete(id) == 1

// DAO SQL
DELETE FROM health_records WHERE id = :id
```

A stale Edit read is ignored if the user has already closed that editor. Saved edit drafts include their id and original values, so restoring the form still targets the original record and detects unsaved changes.

> **Say it aloud**
> **Why not delete by date?**
> Several records can share a date. Deleting by id selects only the intended entry.

Try it: Create two same-day records, edit one and delete the other. Show that row count and totals change as expected, without editing the wrong entry.

22. Goals, progress and database migration

DailyGoals stores configurable steps, calorie and water targets. The application uses id 1 for the settings row and validates positive whole-number targets. A missing row uses defaults in code; Save uses Room Upsert.

```
// Progress principle used by DailyGoals
val ratio = value.toDouble() / goal.coerceAtLeast(1)
val progress = ratio.coerceIn(0.0, 1.0).toFloat()

// Example: 1000 ml / 2000 ml = 0.5 = 50%
```

The progress bar stops at 100%; the displayed actual total may exceed the target. These are personal targets, not automatic exercise prescriptions.

Why database version 2?

Version 1 had health_records. Version 2 adds daily_goals. MIGRATION_1_2 creates the new table without dropping the old one. HealthMateApplication registers this migration in the Room builder. Exported JSON schemas describe the versions; exporting alone does not execute a migration.

A migration test builds a version-1 database containing a health record, opens it through the new Room database, checks the record remains and verifies goals can be saved. Destructive migration fallback is not enabled.

Open in Android Studio: app/src/main/java/com/navod718/healthmate/core/database/HealthDatabase.kt
Open verified source snapshot on GitHub

Say it aloud
Why not just increase the version number?
Changing the number tells Room a schema change exists; it does not describe how to preserve and transform old data. We must provide an appropriate migration.

23. Drafts: rotation, process death and limits

A configuration change, such as rotation, normally recreates the Activity. A ViewModel can survive that recreation. If Android destroys the process, the ViewModel itself is gone; small saved state can help recreate the screen later.

Mechanism	HealthMate use / limitation
ViewModel	Current UI state and actions across normal configuration recreation. Not durable storage after arbitrary process death.
SavedStateHandle	Small draft fields, edit id/original values, selected tab and filter. Restoration depends on Android saved-state/task behavior.
rememberSaveable	Small composition UI choices such as goals-dialog inputs and optional chart selection.
Room	Committed records and goals survive normal restart. Clearing app data/uninstalling removes local data.

The discard check compares current input to original input, not merely whether a field has ever been touched. A loading Edit form can be closed without pretending that unsaved fields already exist.

The Activity uses creation extras to create a SavedStateHandle in the ViewModel factory. Tests cover activity recreation and reconstructing ViewModels from saved-state values. They do not prove recovery after every force-stop, task removal or device failure.

Say it aloud
Does SavedStateHandle replace a database?
No. It helps restore small transient screen state. Room is responsible for committed health records and goals.

Try it: Enter a draft, rotate the emulator, check values, then test Keep editing and Discard. Distinguish the saved draft from a committed record.

24. Optional PR #3: trends and quick water

Version check
This chapter describes feature/trends-quick-water at fab2f19. PR #3 was not merged into main when checked for this guide. Show these features only from a build that contains them.

The chart covers today and the preceding six calendar dates. Room selects rows within those bounds, groups by date and sums each metric. Repository.observeWeek fills missing dates with zero. The chart can switch Steps/Calories/Water and show a tapped day's exact total.

```
-- Weekly aggregation, formatted SQL
SELECT date, SUM(steps) AS steps,
       SUM(calories) AS calories, SUM(water) AS water
FROM health_records
WHERE date >= :start AND date <= :end
GROUP BY date ORDER BY date ASC;
```

Bars are scaled relative to the largest day in the selected series, not relative to the daily goal. The full date and value are exposed for accessibility. No third-party chart library or schema change is added.

Quick water accepts only 250 or 500 ml. It calls the existing Add path with steps=0, calories=0 and today's date. Each completed tap produces a separate record. Busy guards block overlapping requests; errors allow retry. Edit/Delete in Records remain available.

Open in Android Studio: app/src/main/java/com/navod718/healthmate/feature/health/presentation/WeeklyTrends.kt
Open verified source snapshot on GitHub

Say it aloud
Are the charts stored as extra database rows?
No. They are query results derived from health_records. Missing dates are filled in memory for display; original records remain the source data.

25. Testing: what the evidence actually says

Verified snapshot	JVM tests	Android emulator tests	Evidence
Core records branch / PR #2, 6e99015	15 passed	15 passed	Run 42383247885
Optional trends branch / PR #3, fab2f19	20 passed	19 passed	Run 42388832725

These are recorded successful runs from development, not a fresh rerun on the guide date. Both used Android 10 / API 29 for instrumented tests. Builds and Android lint passed; passing lint does not mean there are no warnings.

• **JVM tests:** fast logic checks under src/test, including validation, drafts, id preservation, missing records and write guards. Some use a fake DAO.

• **Instrumented tests:** src/androidTest on Android, using real Room databases and Compose UI interactions; Espresso operates the native date-picker dialog.

• **Manual checks:** your Mac API 37 emulator review checked visible behavior, saving, totals and restart persistence. That is different from automated API 29 evidence.

Database tests verify order/filter/CRUD/totals, migration, close/reopen persistence and atomic sample loading. UI tests cover Add/Edit/Delete, date filter/clear/no-results, goals, keyboard actions and activity recreation. Optional tests add weekly boundaries/zero days/large totals and chart/water interactions.

Core CI evidence | Optional-features CI evidence

Say it aloud
Do passing tests prove no bugs?
No. They support specific behaviors under tested conditions. Physical devices, other Android versions, accessibility settings and unusual failures still need consideration.

26. Git, GitHub and continuous integration

Git tracks source changes locally. **GitHub** hosts the repository and collaboration tools. A commit is a snapshot with an explanation; a branch is a named line of work; a pull request proposes merging changes.

Project item	What to say
Navod718/healthmate	The public repository used for this app.
PR #1	Person 1 Add/Dashboard, drafts and goals; merged.
PR #2	Person 2 history/filter/Edit/Delete/samples; merged.
PR #3	Optional trends/quick water; open at guide preparation.
GitHub Actions	Automates build, unit tests, lint and emulator checks.

The workflow triggers for main pushes and pull requests. Explicit Android SDK packages avoid an obsolete setup default. A concurrency group avoids duplicate simultaneous runs for a PR. It uploads a debug APK and test reports as artifacts.

```
# Commands to understand; do not run blindly on another branch
git status
git log --oneline -5
git switch branch-name
git pull
```

KSP generates Room implementation files under app/build/generated. Read them if useful, but change the entity/DAO source rather than hand-editing generated output.

A merge includes changes in the target branch. It does not copy those updates into an old ZIP download on your Mac. Download the correct branch again or pull in an actual Git checkout.

Say it aloud
Why use meaningful commits?
They show progressive development, explain decisions and make review or troubleshooting easier. Avoid claiming that an AI-created commit proves you personally wrote every line.

27. Build settings, APKs and release signing

Open in Android Studio: app/build.gradle.kts
Open verified source snapshot on GitHub

Setting	Actual value / meaning
minSdk	26: minimum Android API supported by this build.
compileSdk	35: APIs available while compiling the app.
targetSdk	35: Android behavior/compatibility target declared by the app.
applicationId	com.navod718.healthmate: installed app identity.
JVM target / CI JDK	17: bytecode target / Java environment used for builds.
Room database version	2: schema version, separate from app versionCode.

An API 37 emulator may run an app compiled/targeted at API 35. compileSdk is not a maximum allowed device version. The project is Android-only; its APK cannot install on an iPhone.

A debug APK is convenient for development and uses a debug signing key. The coursework asks for a release APK: generate a properly signed release build with your own key. Keep keys/passwords out of Git. Installing an update normally requires compatible app identity and signing certificate; different debug keys can cause an install conflict.

```
./gradlew testDebugUnitTest lintDebug assembleDebug
./gradlew connectedDebugAndroidTest
```

The first command checks/builds without a running device; the second requires a connected emulator/phone. The guide itself does not create a signed release or upload final coursework deliverables.

28. Errors, privacy and honest limitations

Good answers explain what the app handles and where its limits are. Exception handling is not evidence that failure can never occur.

• **Invalid input:** field/form errors prevent a write.

• **Read failure:** screen error state and Retry.

• **Write failure:** saving state is cleared; input is retained where useful. Delete errors offer retry/cancel.

• **Missing record:** report it; never silently insert a replacement for a failed update.

• **Unsaved changes:** confirm before discarding.

• **Sample loading:** confirmation plus an empty-database transaction guard; it never clears existing records first.

Records are in app-local SQLite storage. The manifest declares no internet permission and sets allowBackup=false. Do not call the app fully secure or encrypted: it has no added database encryption, user authentication or audited health-data protection. Device/OS backup and transfer behavior also deserves review before real sensitive-data use.

There is no cloud sync, remote API, iOS build, sensor step detection or guaranteed reminder/background service. Date filtering is exact-date, not a date range. These are fair future-work areas, not features to invent during the viva.

Say it aloud
What would you improve next?
I would prioritise broader device/accessibility testing, then a deliberate export/backup and privacy design. For automatic step tracking I would first design sensor permissions and data handling rather than claiming manual values are sensor measurements.

29. Your guided demonstration

Practise this in 4-6 minutes. Use the actual version you plan to show. Use a safe development build and record the starting totals so you can predict changes.

Step	Action and explanation
1: Overview	Show Dashboard/Records. Explain the stack, local storage and manual inputs.
2: Validate	Open Add, try blank fields and all-zero values. Explain numeric keyboard versus validation.
3: Create	For today, save 1200 / 60 / 500. Predict and show the increase in totals.
4: Persist	Stop and run again without uninstalling. Show that committed records remain.
5: Read/filter	Open Records, choose today and clear the date filter.
6: Update	Edit your test entry to 1500 / 80 / 750. Show row identity/count and new totals.
7: Delete	Open Delete, Cancel once, then confirm. Predict the decrease.
8: Draft/goals	Show Keep editing/Discard, activity rotation and one goal change.
9: Optional extras	Only with PR #3 code: log 250 ml, select chart metric/day, then show the new row.

For a clean numerical demo, take the baseline totals and add/subtract your test entry. Existing records mean the dashboard will not necessarily start at zero. Yesterday's entries do not contribute to today.

Try it: Give this demonstration twice without reading the table. On the second run, narrate which file/layer handles each action. Keep screenshots of real outcomes for report evidence.

30. Small live-coding exercises

Use a temporary local branch or a duplicate project. These are practice tasks, not instructions to alter the assessed app blindly. Revert experiments after checking them.

1. Change UI wording

Find the heading in DashboardScreen in Screens.kt and change its String. Explain that this changes presentation, not the stored schema. Rebuild and visually confirm.

2. Change a default goal

Change the DailyGoals constructor default water from 2000 to 2500. Existing saved goal rows still win; test on a database without a goals row or use the settings dialog. This is a code-default change, not a schema-default migration.

3. Diagnose an invalid input

Use 2147483648 steps. Point to toIntOrNull in RecordValidator and explain overflow. Do not remove validation to make the error disappear.

4. Reverse history order in a practice branch

Change the history SQL to ORDER BY date ASC, id ASC. Run the history ordering test: the newest-first expectation should fail. Explain the failure, then restore DESC.

5. Add a new persistent field: explain the plan

For sleepMinutes, discuss entity, input/validation, DAO/repository, state/UI, schema version, migration, old-row defaults and tests. Adding a TextField alone does not store it. Do not attempt a rushed schema change during the viva without a plan.

6. Trace a failure

In a fake DAO test, simulate a failed write. Check input remains, busy becomes false and retry works. Explain why a fake is useful and why real Room integration tests are also needed.

31. Viva questions 1-10: foundations

1. What problem does HealthMate solve?

It lets a user manually maintain dated wellness records, review today's totals and manage history locally. It is a tracking tool, not automatic sensing or diagnosis.

2. Is your app native or cross-platform?

Native Android. It uses Kotlin, Jetpack Compose and Android Room. This repository does not provide an iOS build.

3. What is Kotlin?

The programming language used for our logic and Compose UI definitions. Android Studio is the IDE, not the language.

4. What is Compose?

A declarative Android UI toolkit. Composable functions describe screens from state, and observed state changes can trigger recomposition.

5. What is Room?

An Android persistence library over SQLite. We define entities, a DAO and a RoomDatabase; Room generates implementation code and validates supported mappings/queries during compilation.

6. What is an emulator?

A virtual Android device running on a computer. It lets us install and exercise the APK; it does not replace testing every real-device condition.

7. What is an APK?

An installable Android application package containing compiled app code and resources. A debug APK and a signed release APK serve different delivery purposes.

8. What are val and var?

val prevents reassignment of that reference/value; var permits it. A mutable object referenced by val can still change internally.

9. What is a data class?

A Kotlin class for holding values with generated equality/copy behavior. HealthRecord represents a database row; state data classes represent screen state.

10. Why are form numbers initially Strings?

Text fields receive text, including blank or invalid input. We validate it first, then convert to Int before a database write.

32. Viva questions 11-20: architecture and state

11. Explain your architecture.

Compose screens send events to HealthViewModel. It coordinates state and calls HealthRepository, which uses HealthRecordDao and Room/SQLite. Observable results return as UI state.

12. Why have a repository?

It gives the ViewModel an application-facing data boundary and centralises write validation and operations. It keeps DAO details out of screen code.

13. What is the ViewModel responsible for?

Screen state and actions: validation, busy/loading state, coordinating writes, saved drafts and user feedback. It does not draw widgets or replace Room persistence.

14. What is StateFlow?

A state holder with a current value and observable updates. The UI collects it, allowing new state to be reflected in Compose.

15. What is recomposition?

Compose reruns relevant UI functions when the state they observe changes. It is not the same as restarting Android or the entire app process.

16. What does flatMapLatest do in filtering?

When the selected filter changes, it switches to the latest query stream and cancels collection of the previous one.

17. What is a coroutine?

A structured asynchronous task that can suspend without blocking its current thread. Room's suspend/Flow APIs perform database work using supported asynchronous execution.

18. Does suspend mean background thread?

No. It permits suspension. The implementation or dispatcher determines where actual work executes; arbitrary blocking work is not made safe just by marking a function suspend.

19. How do you avoid duplicate writes?

The UI disables controls while busy, and ViewModel actions check the busy flag before launching another write. Sample loading also has a transaction guard.

20. Do you use Hilt or NavController?

No. Dependencies are manually supplied by an Application/repository and ViewModel factory. Navigation here is controlled by tab/editor state in HealthMateApp.

33. Viva questions 21-30: data and CRUD

21. What are your tables?

health_records stores id, date, steps, calories and water. daily_goals stores the singleton settings row used by the app for personal targets.

22. Why is date not the primary key?

Multiple entries are allowed on one date. A generated id uniquely identifies each entry for editing and deletion.

23. What does CRUD mean?

Create inserts a record; Read queries history/totals; Update changes an existing id; Delete removes a confirmed record.

24. How does Add work?

The form sends input to the ViewModel, validation runs, the repository converts valid text, and a suspend DAO insert persists a new entity. Observers then update the UI.

25. How does Edit avoid duplicates?

EditorState retains the existing id, and the repository uses @Update. A zero affected-row count is reported instead of falling back to insertion.

26. How does Delete work?

A target is selected first; Cancel performs no write. Confirm deletes by id. On success, reactive queries refresh the list and dashboard.

27. How do you calculate today's totals?

A query filters by the device's current local date and uses SUM for each metric. COALESCE supplies zero if there are no matching rows.

28. Why use YYYY-MM-DD?

It is a canonical, zero-padded date representation suitable for consistent equality and chronological text ordering. LocalDate parsing helps validate calendar dates.

29. What prevents sample duplicates?

A transaction checks that the table is empty before inserting all four samples. Concurrent calls cannot independently pass the guard and both commit copies.

30. What is a migration?

A defined transformation between schema versions. Our 1-to-2 migration creates daily_goals while preserving existing health_records; it is registered in the database builder.

34. Viva questions 31-40: behavior and testing

31. What validation do you perform?

Required fields, canonical valid/non-future dates, digits-only whole Int values and at least one positive metric. Add and Edit share the same validator.

32. Why validate in the repository too?

It protects the write boundary if a caller reaches it without the normal form. UI messages improve usability; boundary validation protects the operation.

33. What happens on rotation?

A ViewModel normally survives configuration recreation. SavedStateHandle also records small draft/edit/tab/filter state for supported restoration; Room keeps committed records.

34. What is the difference between goals and actual totals?

Goals are editable positive targets; totals come from stored entries. A progress bar is capped at 100%, but the displayed actual total remains uncapped.

35. What happens if the record was already deleted?

Opening a missing id reports the missing record. Saving a now-missing id keeps useful input and reports that it was not saved; it does not create a new row.

36. Which tests run without an Android device?

The JVM tests under src/test, such as validation and ViewModel tests with a fake DAO and controlled coroutine dispatching.

37. Which tests use an emulator?

The instrumented tests under src/androidTest, including real Room persistence/migration and Compose/native date-picker UI flows.

38. How many tests passed?

For merged-core evidence, 15 JVM plus 15 emulator tests. For optional PR #3, 20 JVM plus 19 emulator tests. State the snapshot and show its CI evidence.

39. What does lint prove?

It reports static analysis issues. Passing lint is useful but does not prove runtime correctness, full accessibility or absence of warnings.

40. Describe a real issue and fix.

The first UI run could not find the Extended FAB label in the merged semantics tree. The test selector was corrected to the unmerged tree and the suite rerun successfully; this was a test-selector issue.

35. Viva questions 41-50: extras and judgement

41. How are weekly trends calculated?

In optional PR #3, Room groups a bounded seven-day range by date and sums metrics. The repository supplies zero entries for dates with no rows.

42. What do chart bar heights represent?

Relative values within the selected metric series, scaled against its largest day. They are not daily-goal percentages. Tapping a day shows its exact amount.

43. How does quick water work?

Optional buttons log 250 or 500 ml using the existing Add path, with zero steps/calories and today's date. Each successful tap creates an editable/deletable record.

44. Is this data encrypted?

The app uses app-local SQLite storage but adds no database encryption layer. I would not claim audited security or health-data compliance.

45. Why is there no login?

The implemented scope is an offline, single-device tracker. Multi-user login and cloud sync would require additional identity, networking and privacy design.

46. Can this APK run on an iPhone?

No. This is an Android build. An iOS implementation would require an appropriate separate platform strategy.

47. Why do older records not appear in today's totals?

Totals intentionally filter to the current local date. Older entries remain in Records and can be filtered by their own dates.

48. What is the difference between a branch and main?

A feature branch isolates proposed work. main contains merged work. An open feature PR is not automatically included in a main ZIP/build.

49. How did you use AI or other help?

Give your actual account of assistance, changes, testing and learning. Do not claim independent authorship of assisted work. Follow the assessment's acknowledgement rules.

50. What if you do not know an answer?

Say what you understand, identify the relevant layer/file and reason through an example. If uncertain, say so and verify rather than inventing a framework, feature or result.

36. Mock viva: rehearse a real conversation

Ask a friend to act as examiner for 15 minutes. Keep the app and source open. The interviewer should interrupt politely with "show me where" and "what if it fails?"

Time	Examiner prompt	What you should demonstrate
0-2 min	What does your app do?	A clear overview, stack and actual platform.
2-5 min	Show one record being saved.	Validation, Add flow, DAO and dashboard update.
5-8 min	Explain Edit/Delete and same-day entries.	Id preservation, confirmation and correct totals.
8-10 min	What happens on rotation or failure?	Draft state, busy/finally and error feedback.
10-12 min	Explain schema change and testing.	Migration source, test name and CI evidence.
12-15 min	Show a small code change or discuss extras.	Read code, predict impact and test logically.

Score your readiness, not your confidence

For each answer give yourself: **0** cannot explain; **1** definition only; **2** definition plus correct file/example; **3** can demonstrate and explain an edge case. Revisit every zero or one before learning more optional features.

An honest opening you can adapt

"HealthMate is a native Android tracker. I used development assistance and have been studying and testing the implementation. I can explain the data flow and demonstrate the implemented features." Adapt this only to your actual experience and the required declaration; it is not a claim of authorship supplied by this guide.

37. Five-day revision and readiness checklist

Day	Study and practice	Output
1	Read foundations/Kotlin and run the app.	Explain the stack and five Kotlin expressions.
2	Trace Add and validation through all layers.	Draw the architecture from memory.
3	Study DAO, Edit/Delete, goals and migration.	Predict totals and demonstrate id preservation.
4	Study state/coroutines/testing/Git; run exercises.	Explain rotation, errors and one real test.
5	Complete two mock vivas and the demo.	A short list of weak points to revisit.

Before you say you are ready

- I can explain language versus UI toolkit versus database library.
- I can locate MainActivity, ViewModel, repository, DAO and database.
- I can trace Add, Edit, Delete and date filtering without reading a script.
- I can explain why ids, Long totals, validation and migration matter.
- I can distinguish ViewModel, SavedStateHandle and Room persistence.
- I can show the actual test run and explain its limits.
- I know which branch/build contains optional trends and quick water.
- I can describe assistance and contributions truthfully.

If the viva is tomorrow
Spend your limited time on the overview, architecture, CRUD, validation, database design, state and evidence. Being able to explain six core flows well is better than guessing about many advanced topics.

38. Glossary: essential terms A-M

Term	Meaning in plain language
Activity	Android component hosting the screen/window.
Aggregate	A summary such as SUM or COUNT.
Annotation	Metadata beginning with @ for frameworks/tools.
APK	Installable Android application package.
API level	Number identifying Android platform API capabilities.
Architecture	Organisation of responsibilities and dependencies.
Callback	Function supplied to run after an event.
CI	Continuous integration: automated checks/builds on source changes.
Composable	Function used to describe Compose UI.
Coroutine	A structured task that may suspend without blocking.
CRUD	Create, Read, Update and Delete.
DAO	Data Access Object: database operation contract.
Entity	Class mapped to a database table.
Flow	A stream of emitted values over time.
Gradle	Build and dependency automation system.
Idempotence	Repeated operation has the same effect as one; not every app action has this property.
Index	Database structure supporting some lookups/sorts, with maintenance costs.
KSP	Kotlin Symbol Processing; used for Room code generation.
Lifecycle	How an Android component starts, becomes visible, stops and is destroyed.
Migration	Defined transition between database schema versions.

39. Glossary: essential terms N-Z

Term	Meaning in plain language
MVVM	Model-View-ViewModel separation.
Native	Built for the platform's APIs/toolkit in this project.
Nullable	A type whose value may be null.
Persistence	Data remaining beyond a running screen/process.
Primary key	Column(s) uniquely identifying a row.
Pull request	Proposal to merge one line of source work into another.
Query	A database request expressed here in SQL.
Recomposition	Compose updating relevant UI from changed observed state.
Repository	Application-facing boundary for data operations.
Room	Android library mapping entities/DAOs to SQLite persistence.
SavedStateHandle	ViewModel-associated storage for small restorable screen state.
Schema	The structure of tables, columns and related database metadata.
SQLite	Embedded relational database engine.
StateFlow	Observable state with a current value.
Suspend	Function can pause/resume; not a dispatcher guarantee.
Transaction	Atomic group of database operations with controlled execution.
Unit test	Focused logic test; project JVM tests do not require an emulator.
Upsert	Insert or update based on conflict/identity handling.
ViewModel	State/action holder designed for UI lifecycle patterns.
Validation	Checking data against required rules before accepting it.

40. Official references and final reminders

Use official documentation to clarify unfamiliar terms. These are learning references, not a substitute for reading your own project. Reference links are provided directly; this guide does not claim to reproduce the latest documentation verbatim.

Kotlin basics
https://kotlinlang.org/docs/basic-syntax.html

Kotlin null safety
https://kotlinlang.org/docs/null-safety.html

Jetpack Compose
https://developer.android.com/compose

Android architecture guidance
https://developer.android.com/topic/architecture

ViewModel
https://developer.android.com/topic/libraries/architecture/viewmodel

Saved UI state
https://developer.android.com/topic/libraries/architecture/saving-states

Room persistence
https://developer.android.com/training/data-storage/room

Room migrations
https://developer.android.com/training/data-storage/room/migrating-db-versions

Kotlin coroutines on Android
https://developer.android.com/kotlin/coroutines

Compose testing
https://developer.android.com/develop/ui/compose/testing

Android app signing
https://developer.android.com/studio/publish/app-signing

Git tutorial
https://git-scm.com/docs/gittutorial

Coursework still needs its own deliverables

A viva guide does not replace the required technical report, signed release APK, narrated video, sharing links, Harvard references or actual test evidence in the submission. Follow the final lecturer/ELMS instructions and the required AI acknowledgement.

Prepared from the supplied CW1 brief, the project source snapshots identified in this guide, historical CI evidence, and the documented manual-review conversation. Main and PR status were checked on 10 November 2026. No new app tests were run solely to create this PDF.